United International University (UIU)
Dept. of Electrical and Electronic Engineering (EEE)
Course No. EEE 312
Course Title: Digital Signal Processing Lab

Lab Sheet 5
Discrete Fourier Transform (DFT) using MATLAB

Prerequisite

Before starting this section, students should know..

1. A fundamental knowledge on MATLAB
2. A theoretical knowledge on DFT and FFT.

Outcomes

After finishing this lab students should be able to.....

1. investigate the Discrete Fourier Transform
2. learn how to implement the operation using MATLAB
3. learn how to analyze discrete-time signal using DFT
4. learn how to synthesize discrete-time signal using IDFT
5. analyze some real life problems involving DFT.

Outlines of this Lab

1. Lab Session 5.1: DFT and it's MATLAB Implementation
2. Lab Session 5.2: FFT and it's MATLAB Implementation
3. Lab Session 5.3: Practical Applications of DFT
4. In Lab Evaluation
5. Home work

Lab Session 5.1:DFT and it's MATLAB Implementation

Theoretical Background

DFT: The Discrete Fourier Transform of an N-point sequence is given by

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}, \qquad 0 \le k \le N-1 \qquad (1)$$

where $W_N = e^{-j\frac{2\pi}{N}}$. Note that the DFT $X(k)$ is also an N-point sequence, that is, it is not defined outside $0 \le k \le N-1$.

IDFT: The inverse discrete Fourier transform (IDFT) of an N-point DFT $X(k)$ is given by (once again $x(n)$ is not defined outside $0 \le n \le N-1$)

Prepared by BKM Mizanur Rahman, updated by MK Masukur Rahman, Dept of EEE, UIU

$$x(n) = \frac{1}{N}\sum_{k=0}^{N-1} X(k) W_N^{-nk}, \qquad 0 \le n \le N-1 \qquad (2)$$

In MATLAB, an efficient implementation of the both transforms, DFT and IDFT, would be to use a matrix-vector multiplication for each of the relations in (1) and (2).If $x(n)$ and $X(k)$ are arranged as row vectors **x** and **X**, respectively, then from (1) and (2) we have

$$\mathbf{X} = \mathbf{x}\mathbf{W}_N \qquad (3)$$

$$\mathbf{x} = \frac{\mathbf{1}}{N}\mathbf{X}\mathbf{W}_N^* \qquad (4)$$

where $\mathbf{W}_N$ and $\mathbf{W}_N^*$ are a *DFT matrix* and *its conjugate*, respectively. The following MATLAB functions implement the above equations.

Custom Functions

```
function [Xk] = dft(xn, N)
% Computes Discrete Fourier Transform
% ------------------------------------------------
% [Xk] = dft(xn, N)
% Xk = DFT coeff. Array over 0 <= k <= N-1
% xn = N-point finite-duration sequence
%  N = Length of DFT
%
n = [0:1:N-1];       % row vector for n
k = [0:1:N-1];       % row vector for k
WN = exp(-j*2*pi/N);     % Wn factor
nk = n'*k;           % creates a NxN matrix of nk values
WNnk = WN .^ nk;         % DFT matrix
Xk = xn * WNnk;      % row vector for DFT coefficients
% End of function
```

```
function [xn] = idft(Xk, N)
% Computes Inverse Discrete Fourier Transform
% ----------------------------------------
% [xn] = idft(Xk, N)
% xn = N-point sequence over 0 <= n <= N-1
% Xk = DFT coeff. Array over 0 <= n <= N-1
%  N = Length of DFT
%
n = [0:1:N-1];       % row vector for n
k = [0:1:N-1];       % row vector for k
WN = exp(-j*2*pi/N);     % Wn factor
nk = n'*k;           % creates a NxN matrix of nk values
WNnk = WN .^ (-nk); % IDFT matrix
xn = (Xk*WNnk)/N;        % row vector of IDFT values
% End of function
```

Example 5.1:

Let x(n) be a 4-point sequence defined as, x = [1, 0.5, 0.2, 0.1] for 0 ≤n ≤ 3, zero otherwise.

Compute 4-point DFT of x(n)

Solution:

MATLAB Code:

```
x = [1, 0.5, 0.2, 0.1];  n=0:3; N = 4;
X = dft(x,N);
magX = abs(X), phaX = angle(X)*180/pi

subplot(211);stem(n,magX,'k');title('Magnitude')
subplot(212);stem(n,phaX,'k');title('Angle'))
```

Output:

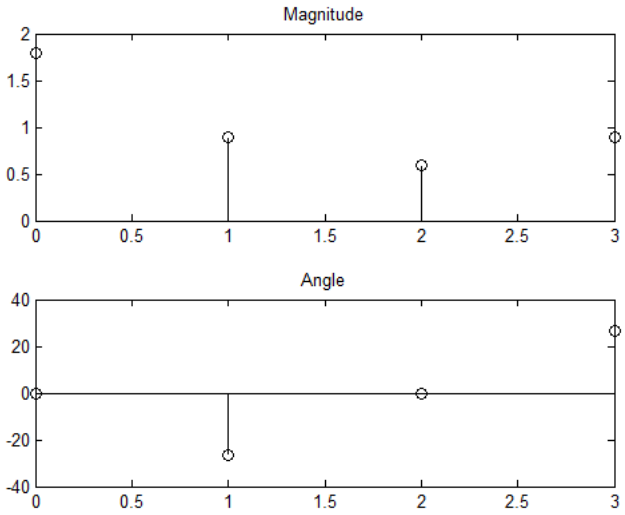
magX =

1.8000 0.8944 0.6000 0.8944

phaX =

0 -26.5651 -0.0000 26.5651

Figure:



MATLAB Implementation of Inverse Discrete Fourier Transform:

Example 5.2: find the inverse DFT X(k) in above example.

Solution:

MATLAB Code:

```
idft(X,4)
```

Output:

ans =

1.0000 - 0.0000i 0.5000 + 0.0000i 0.2000 + 0.0000i 0.1000 + 0.0000i

Which is same as x = [1, 0.5, 0.2, 0.1];

Zero Padding in DFT:

Example 5.3: Apply the zero-padding operation, which is a technique that allows us to sample at dense (or finer) frequencies, to get an 8-point sequence and then compute its DFT by using the following MATLAB commands.

Solution:

MATLAB Code:

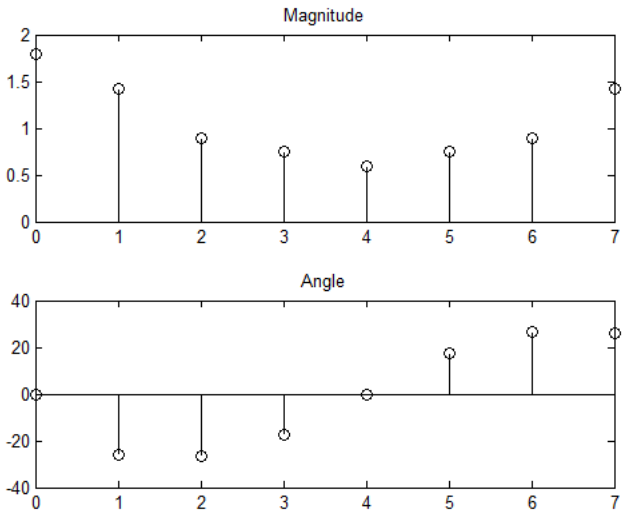
```
x = [1, 0.5, 0.2, 0.1, zeros(1,4)]; N = 8; X = dft(x, N);
magX = abs(X), phaX = angle(X)*180/pi
n=0:7;
subplot(211);stem(n,magX,'k');title('Magnitude')
subplot(212);stem(n,phaX,'k');title('Angle')
```

Output:

At Command window:

```
magX =
   1.8000    1.4267    0.8944    0.7514    0.6000    0.7514    0.8944    1.4267
phaX =
         0  -25.9487  -26.5651  -17.3651   -0.0000   17.3651   26.5651   25.9487
```

Figure:



Can you see that DFT (magnitude) is symmetric about middle point (excluding first point)?

You have noticed similarity in frequency response between original and zero padding

versions. Now try again with a new value of x=[1, 1, 1, 1] and see the difference again.

Example 5.4:

For the sequence $x(n) = \cos(0.2\pi n) + \cos(0.6\pi n)$, use the following MATLAB commands to determine the 30-point DFT of $x(n)$.

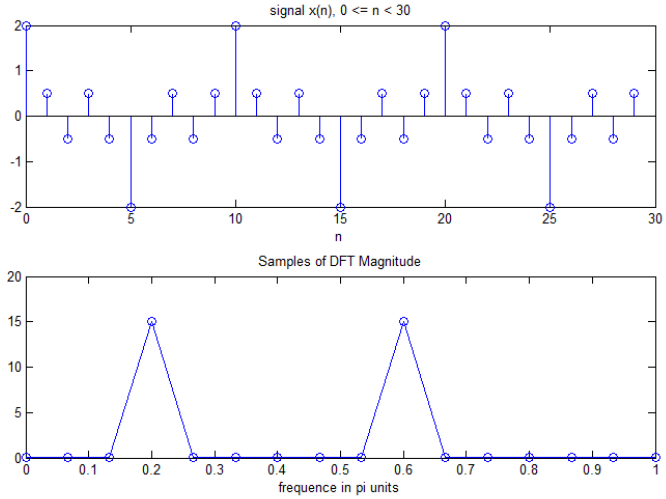
Solution:

MATLAB Code:

```
N=30; n = [0:1:N-1]; y = cos(0.2*pi*n)+cos(0.6*pi*n);
subplot(2, 1, 1); stem(n, y);
title('signal x(n), 0 <= n < 30');
xlabel('n');

Y = dft(y, N);  M = floor(N/2)+1; % middle of DFT, as symmentric
magY = abs(Y(1:1:M)); k =  0:1:M-1; w = 2*pi/N*k;
subplot(2, 1,  2); plot(w/pi, magY , '-o');
title('Samples of DFT Magnitude');
xlabel('frequence in pi units');
```

Output:

Figure:



Circular folding: If an N-point sequence is folded, the result x(-n) would not be an N-point sequence, and it would not be possible to compute DFT. Therefore we use the modulo-N operation on the argument (-n) and define folding by,

$$x\big((-n)\big)_N = \begin{cases} x(0), & n = 0 \\ x(N-n), & 1 \le n \le N-1 \end{cases}$$

This is called circular folding.
The DFT of a circular folding is given by,

$$DFT\left[x\left((-\mathrm{n})\right)_N\right]=\begin{cases}\mathrm{X}(0),\ k=0\\ \mathrm{X}(\mathrm{N}-k),1\le k\le N-1\end{cases}$$

Example 5.5:

Let $x(\mathrm{n})=10(0.8)^n \quad 0\le n\le 10$

(a) Determine and plot $x\left((-\mathrm{n})\right)_{11}$

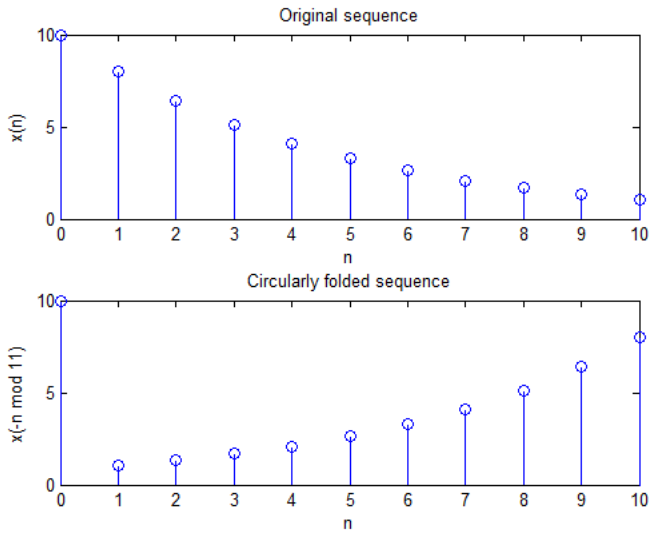
(b) Determine DFT of the circularly shifted sequence.

Solution:

(a) MATLAB Code:

```
n = 0:10; x = 10*(0.8) .^ n;
y = x(mod(-n,11)+1); % Try to see values in "mod(-n,11)"
subplot(2,1,1); stem(n,x); title('Original sequence')
xlabel('n'); ylabel('x(n)');
subplot(2,1,2); stem(n,y); title('Circularly folded sequence')
xlabel('n'); ylabel('x(-n mod 11)');
```

Output:

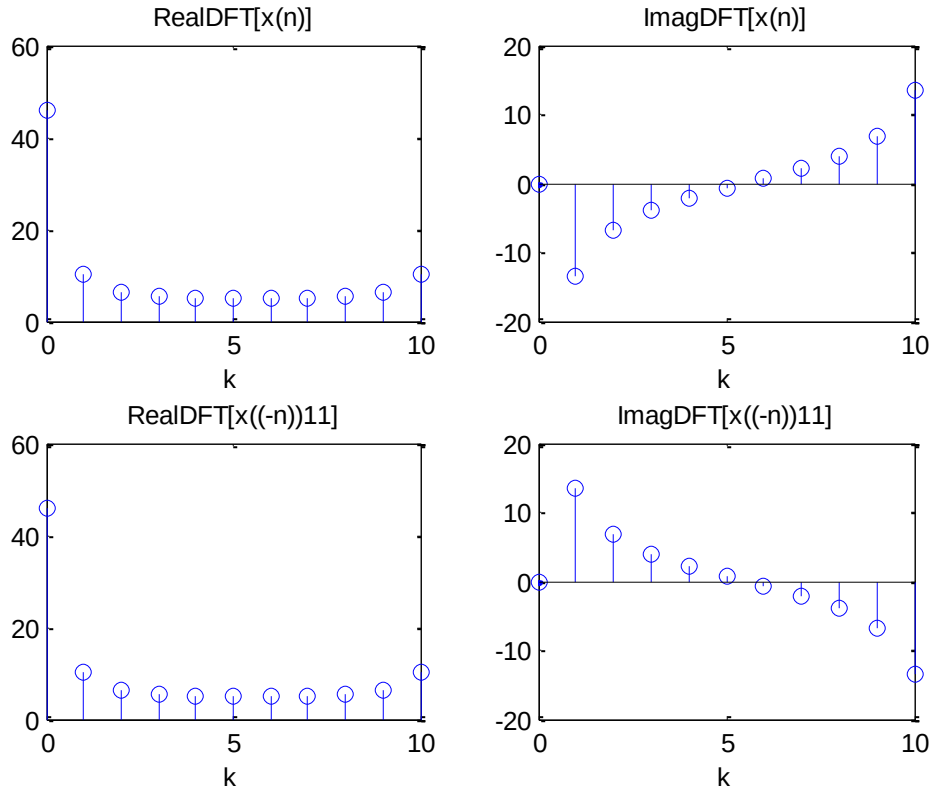


Solution:

(b) MATLAB Code:

```
X = dft(x,11); Y = dft(y,11);
subplot(2,2,1); stem(n,real(X));
title('Real{DFT[x(n)]}'); xlabel('k');
subplot(2,2,2); stem(n,imag(X));
title('Imag{DFT[x(n)]}'); xlabel('k');
subplot(2,2,3); stem(n,real(Y));
title('Real{DFT[x((-n))11]}'); xlabel('k');
subplot(2,2,4); stem(n,imag(Y));
title('Imag{DFT[x((-n))11]}'); xlabel('k');
```

Output:



What changes do you observe in frequency domain due to circular folding operation?

Lab Session 5.2:FFT and it's MATLAB Implementation

The DFT introduced previously is the only tramsform that is discrete in both the time and the frequency domains, and is defined for finite-duration sequences. Although it is a computable transform, the straightforward implementation of it is very inefficient, especially when the sequence length N is large. In 1965 Cooley and Tukey showed a procedure to substantially reduce the amount of computations involved in the DFT. This led to the explosion of application of the DFT, including in digital signal processing area. Furthermore, is also led to the development of other efficient algorithms. All these efficient algorithms are collectively known as Fast Fourier Transform (FFT) algorithms. In an efficiently designed algorithm the number of computations should ne constant per data sample, and therefore the total number of computations should be linear with respect to N. The number of DFT computations for an N-point sequence depends quadratically on N, which will be denoted by the notation

$$C_N = o(N^2) \qquad (1)$$

FFT algorithms can reduce the quadratic dependence on N of the DFT. Theoretically, the number of computations used in the FFT algorithms could be as small as, depending on the radix used in the alogorithm,

$$C_N = o(N \log_2 N) \qquad (2)$$

MATLAB provides a function called **`fft`** to compute the DFT of a vector **`x`**. It is invoked by **`x = fft(x, N)`**, which computes the *N*-point DFT. If the length of **`x`** is less than N, then **`x`** is padded with zeros. If the argument N is omitted, then the length of the DFT is the length of **`x`**. If **`x`** is a matrix, then **`fft(x,N)`** computes the *N*-point of each column of **`x`**.

This **`fft`** function is written in machine language and not using MATLAB commands (i.e., it is not available as a .m file). Therefore it executes very fast. It is written as a mixed-radix algorithm. If *N* is a power of two, then a high-speed redix-2 FFT algorithm is employed. If *N* is not a power of two, then *N* is decomposed into prime factors and a slower mixed-radix FFT algorithm is used. Finally, if *N* is a prime number, then the **`fft`** function is reduced to the raw DFT algorithm.

The IDFT is computed using the **`ifft`** function, having same characteristics as **`fft`**.

Example 5.6:

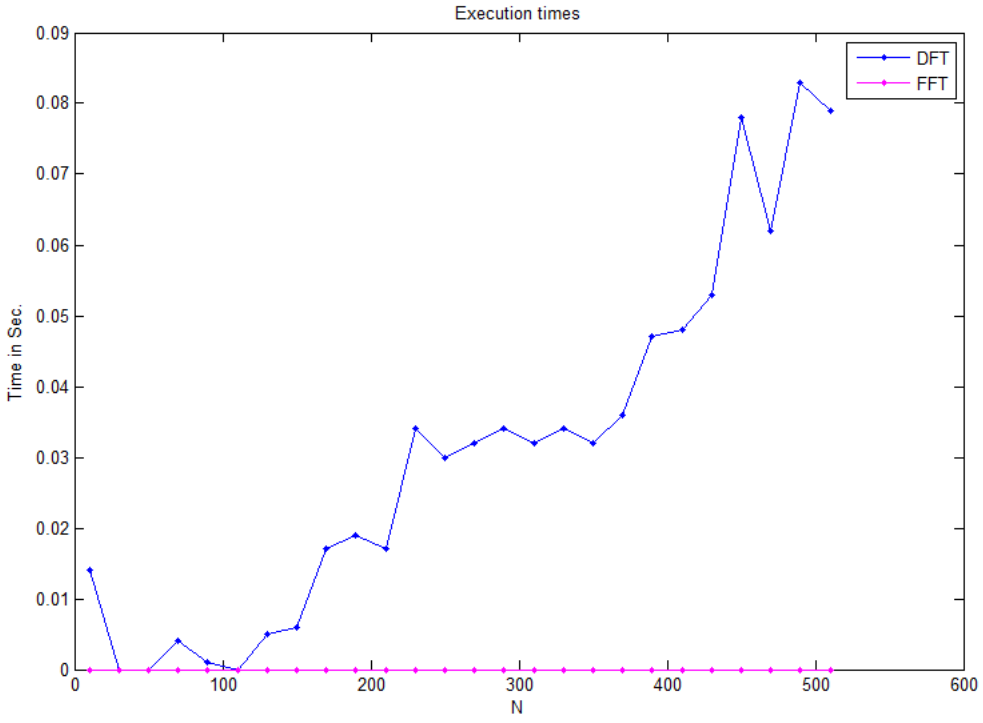
Determine the execution time of the DFT & FFT as N varies from 10 to 512, invoke the following commands and then save the plot.

Solution:

MATLAB Code:

```
clear all;
Nmax = 512; step=20; dft_time=[];fft_time=[];
for n = 10:step:Nmax
    x = rand(1,n);

    t = clock; %Time start
    dft(x, n);
    dft_time = [dft_time  etime(clock, t)];%Time finish & recoreded

    t = clock; %Time start
    fft(x, n);
    fft_time = [fft_time  etime(clock, t)];%Time finish & recoreded

end
n = [10:step:Nmax]; figure(1);

plot(n, dft_time, '.-'); hold on
plot(n, fft_time, '.-m');
xlabel('N'); ylabel('Time in Sec.');
title('Execution times'); legend('DFT','FFT');
```

Output:

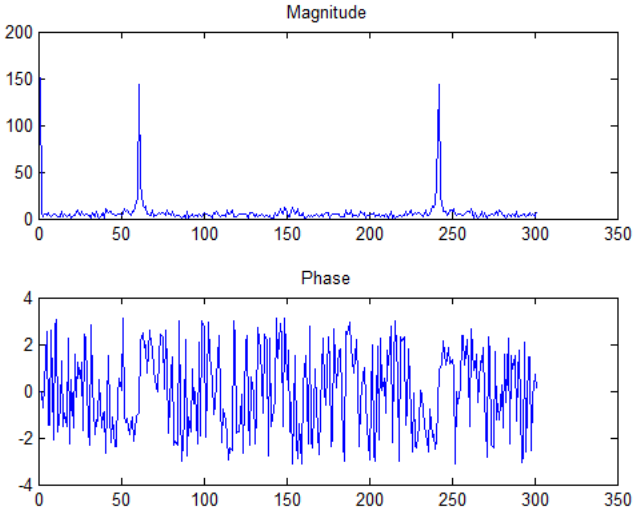


Can you see the difference in time requirements?

Lab Session 5.3:FFT and Practical Applications

FFT of a signal can be computed as follows:

```
t=0:.01:3; f=1/.01;
x = sin(2*pi*20*t);
x1 = x+ rand(size(x));
X1 = fft(x1);
subplot(211); plot(abs(X1)); title('Magnitude');
subplot(212); plot(angle(X1)); title('Phase');
```



Original frequency of sinusoid can be found from FFT as follows:

```
f=1/.01; % sampling frequency
X1=abs(fft(x1));
[v ff1] = max(X1(2:end)); % index of peak frequency in FFT
f1 = (ff1)*f/(length(X1)-1)
```

Here, **fo = f_F * f /(N-1)**
fo = original freq,
f_F = (index of freq in FFT) – 1, (excluding 1st element, DC value)
f = sampling frequency;
N = number of samples (in time series or its FFT)

Example 5.7: FFT of Noisy Data

Sample the sinusoid $x(t) = \cos(2\pi f_0 t)$ with $f_0 = 8$ Hz at $S = 64$ Hz for 4 s to obtain a 256-point sampled signal $x[n]$. Also generate 256 samples of a uniformly distributed noise sequence $s[n]$ with zero mean.

(a) Display the first 32 samples of x[n]. Can you identify the period from the plot? Compute and plot the DFT of x[n]. Does the spectrum match your expectations?

(b) Generate the noisy signal y[n] = x[n] + s[n] and display the first 32 samples of y[n]. Do you detect any periodicity in the data? Compute and plot the DFT of the noisy signal y[n]. Can you identify the frequency and magnitude

of the periodic component from the spectrum? Do they match your expectations?

(c) Generate the noisy signal z[n] = x[n]s[n] (by element-wise multiplication) and display the first 32 samples of z[n]. Do you detect any periodicity in the data? Compute and plot the DFT of the noisy signal z[n]. Can you identify the frequency the periodic component from the spectrum?

Used Custom function:

Collect custom functions Randist and dtplot from server-7

MATLAB Code:

```
N=256;n=0:N-1; f0=8;S=64;F=f0/S;

x=cos(2*pi*n*F);s=randist(x,'uni',0);
figure; subplot(211),dtplot(n(1:32),x(1:32),'.'); title('Original
Signal');
X=fft(x);f=n*S/N;
subplot(223),plot(f,abs(X)), subplot(224),plot(f,angle(X));

y=x+s;
figure; subplot(211),dtplot(n(1:32),y(1:32),'.');
title('Signal+Noise');
Y=fft(y);f=n*S/N;
subplot(223),plot(f,abs(Y)), subplot(224),plot(f,angle(Y));

z=x.*s;
figure; subplot(211),dtplot(n(1:32),y(1:32),'.');
title('Signal*Noise');
Z=fft(y);f=n*S/N;
subplot(223),plot(f,abs(Z)), subplot(224),plot(f,angle(Z));
```

Output:

Figure-1:

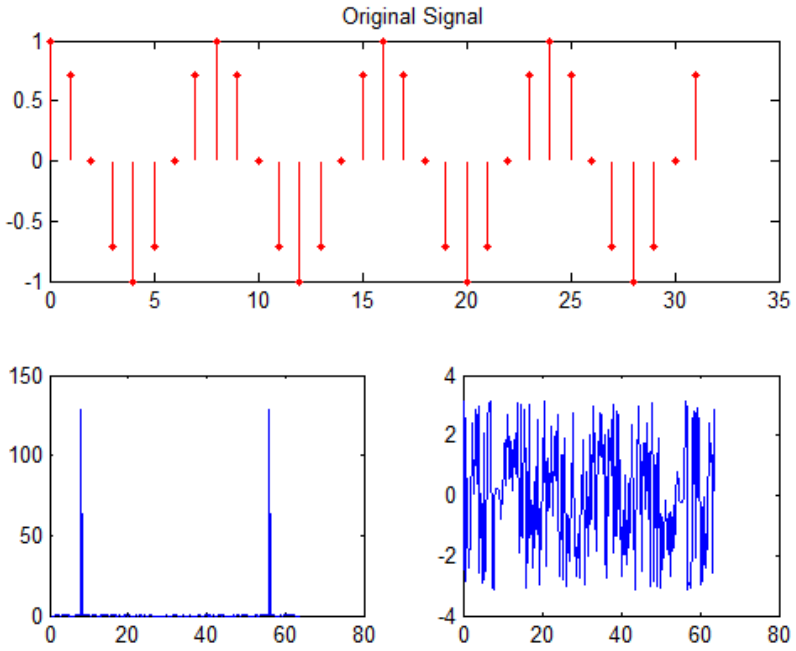


Figure-2:

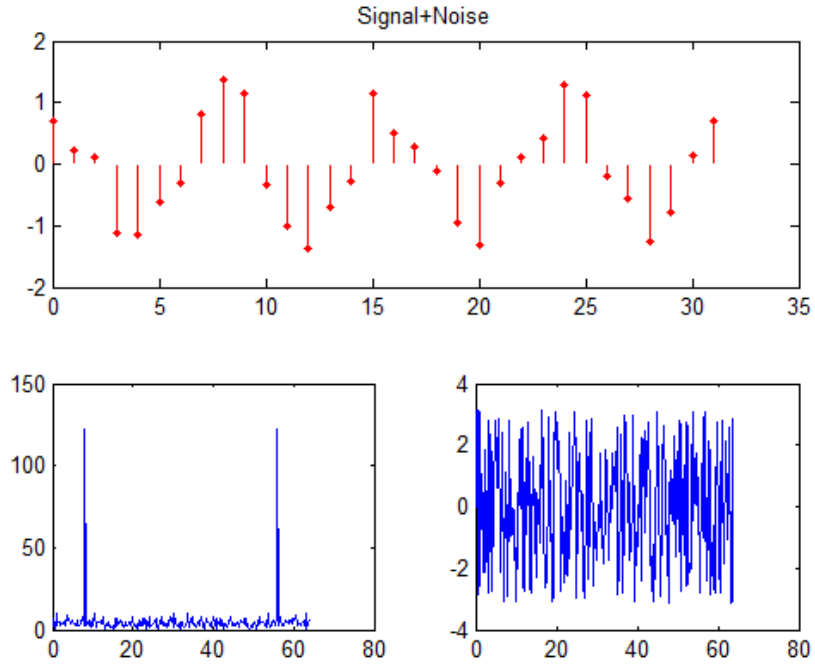
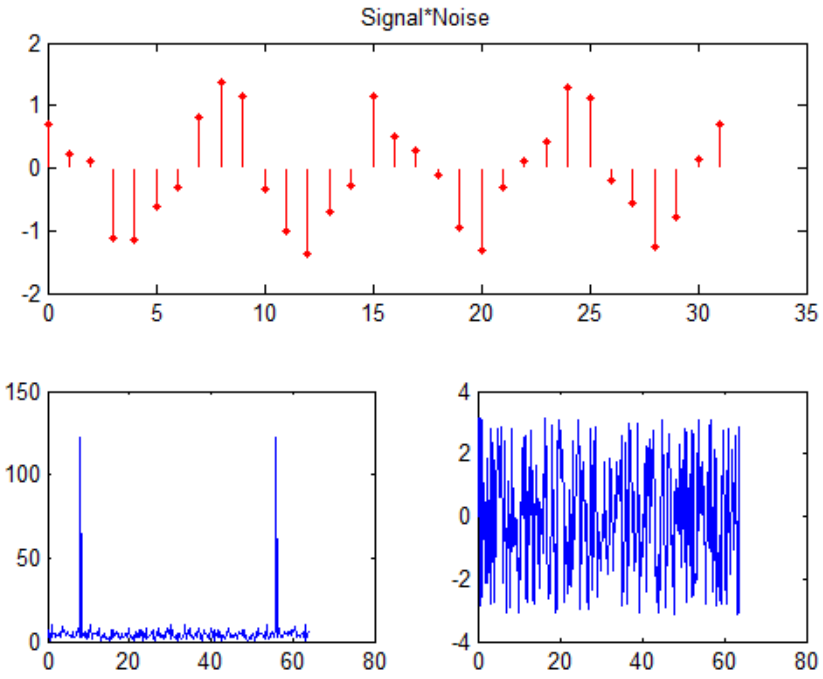


Figure-3:



Comment on the above three figures.

Can you reconstruct the original signal from the noisy one by manipulating the fourier transform?

Try yourself first, then consider the following code:

```
i=find(abs(Y)<25); Y1 = Y;
Y1(i) = 0;
y1 = ifft(Y1);
figure; subplot(211),dtplot(n(1:32),y1(1:32),'.');  title('Signal+Noise');
subplot(223),plot(f,abs(Y1)), subplot(224),plot(f,angle(Y1));
```

Example 5.8: Filtering a Noisy ECG Signal

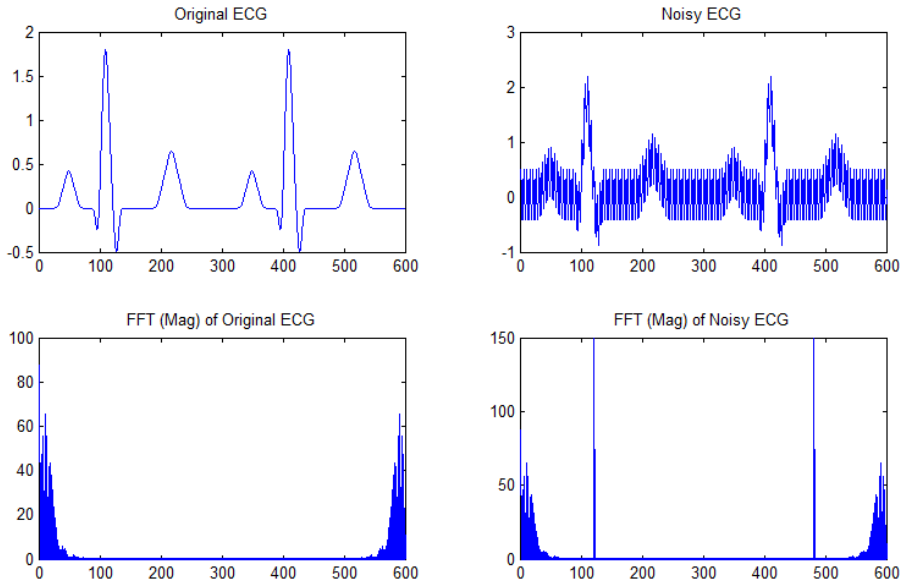
During recording, an electrocardiogram (ECG) signal, sampled at 300 Hz, gets contaminated by 60-Hz hum. Two beats of the original and contaminated signal (600 samples) are provided at server-7 ecgo.mat and ecg.mat. Load these signals into MATLAB (for example, use the command load ecgo). In an effort to remove the 60-Hz hum, use the DFT as a filter to implement the following steps

(a) Compute 600-point DFT of the contaminated ECG signal
(b) Compute the DFT indices that correspond to the 60-Hz signal.
(c) Zero out the DFT components corresponding the 60-Hz signal.
(d) Take the IDFT to obtain the filtered ECG and display the original and filtered signal.
(e) Display the DFT of the original and filtered ECG signal and comment on the differences.
(f) Is the DFT effective in removing the 60-Hz interference?

Solution:

MATLAB code:

```
load ecg; load ecgo;
%place ecg.mat and ecgo.mat in current folder
F1=fft(ecgo); F2=fft(ecg);
subplot(221);plot(ecgo); title('Original ECG');
subplot(222);plot(ecg);  title('Noisy ECG');
subplot(223);plot(abs(F1));  title('FFT (Mag) of Original
ECG');
subplot(224);plot(abs(F2));  title('FFT (Mag) of Noisy
ECG');
```



Home Task: Reconstruct the noise-free ECG signal from the noisy-ECG signal.

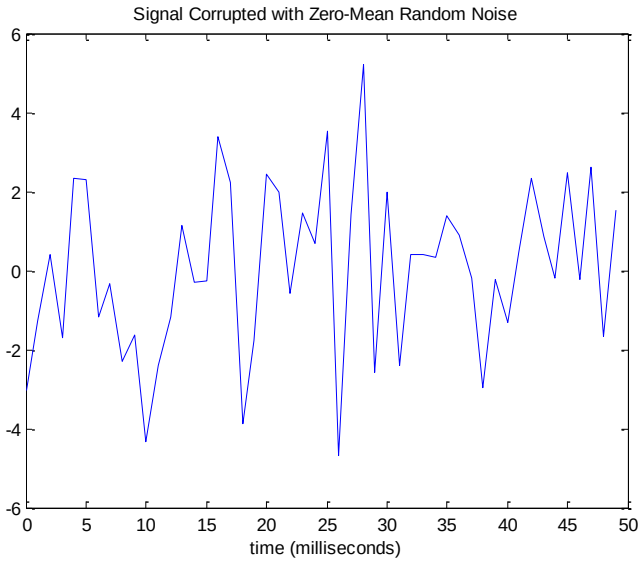
Example 5.9: Finding frequency components of a signal buried in a noisy time domain signal

A common use of Fourier transforms is to find the frequency components of a signal buried in a noisy time domain signal. Consider data sampled at 1000 Hz. Form a signal containing a 50 Hz sinusoid of amplitude 0.7 and 120 Hz sinusoid of amplitude 1 and corrupt it with some zero-mean random noise, Plot the Signal Corrupted with Zero-Mean Random Noise.

Solution:

MATLAB code:

```
Fs = 1000;                      % Sampling frequency
T = 1/Fs;                       % Sample time
L = 1000;                       % Length of signal
t = (0:L-1)*T;                  % Time vector
% Sum of a 50 Hz sinusoid and a 120 Hz sinusoid
x = 0.7*sin(2*pi*50*t) + sin(2*pi*120*t);
y = x + 2*randn(size(t));       % Sinusoids plus noise
plot(Fs*t(1:50),y(1:50))
title('Signal Corrupted with Zero-Mean Random Noise')
xlabel('time (milliseconds)')
```

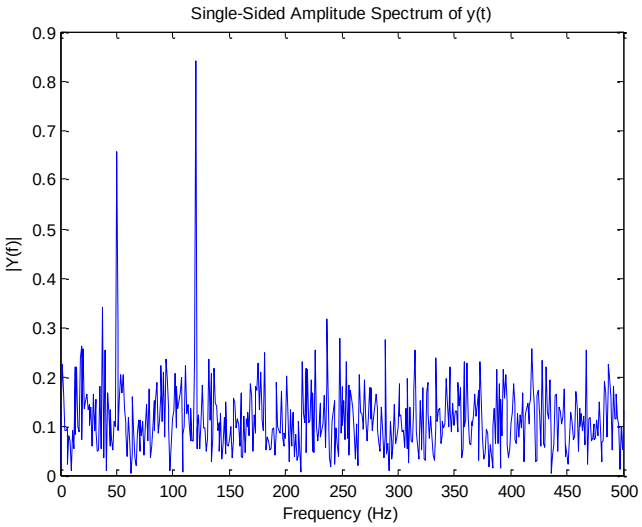


Comment: It is difficult to identify the frequency components by looking at the original signal. Converting to the frequency domain, the discrete Fourier transform of the noisy signal y is found by taking the fast Fourier transform (FFT):

Extension of example 5.10 :Plotting after taking fft

```
Fs = 1000;                      % Sampling frequency
T = 1/Fs;
L=1000;
NFFT = 2^nextpow2(L); % Next power of 2 from length of y

t = (0:L-1)*T;
x = 0.7*sin(2*pi*50*t) + sin(2*pi*120*t);
y = x + 2*randn(size(t));
Y = fft(y,NFFT)/L;
f = Fs/2*linspace(0,1,NFFT/2+1);

% Plot single-sided amplitude spectrum.
plot(f,2*abs(Y(1:NFFT/2+1)))
title('Single-Sided Amplitude Spectrum of y(t)')
xlabel('Frequency (Hz)')
ylabel('|Y(f)|')
```

Output:



Comment: The main reason the amplitudes are not exactly at 0.7 and 1 is because of the noise. Several executions of this code (including recomputation of y) will produce different approximations to 0.7 and 1. The other reason is that you have a finite length signal. Increasing L from 1000 to 10000 in the example above will produce much better approximations on average.

Home Task: Reconstruct the noise-free signal from the noisy signal.

In Lab Evaluation:

ILE 5.1 A 12-point sequence x(n) is defined as

$$x(n) = \{1, 2, 3, 4, 5, 6, 6, 5, 4, 3, 2, 1\}$$

Determine the DFT of x(n). Plot it's magnitude and phase

ILE 5.2 Let $x(n) = (0.5)^n \quad 0 \le n \le 7$

(a) Determine and plot $x((-n))_8$

(b) Determine DFT of the circularly shifted sequence.

Home Work

HW5.1 Correlation is an effective method of detecting signals buried in noise. Noise are essentially uncorrelated.This means that, if we correlate a noisy signal with itself, the correlation will be only due to the signal only.This will exhibit a sharp peak at n=0. Generate two noisy signals by adding noise to a 20Hz sinusoid sampled at Ts=0.01 sec for 1 seconds.

(a) Verify the presence of the signal by ploting correlation of the two noisy signals.
(b) Can you see the presence of periodicity in correlation plot? Where it coming from?
(c) Plot FFT spectrum of the correlation.
(d) Can you see the original sinusoid frequency in the FFT?

HW5.2 During transmission, a message signal gets contaminated by a low-frequency signal and

high-frequency noise. The message can be decoded only by displaying it in the time domain. The contaminated signal x[n] is provided at server-7 as **mystery1.mat.**
Load this signal into Matlab (use the command load mystery1). In an effort to decode the message,try the following procedure and determine what the decoded message says.

(a) Display the contaminated signal. Can you "read" the message?
(b) Display the DFT of the signal to identify the range of the message spectrum.
(c) First zero out the DFT component corresponding to the low-frequency contamination and obtain its IDFT y[n].
(d) Next, zero out the DFT component corresponding to the high-frequency contamination and obtain its IDFT y[n].
(e) Take the IDFT to obtain the filtered signal and display it to decode the message.